International Institute of Medical Science & Technology Council

A Mini Project Report On "**AFFORDABLE HOUSING PRICE ESTIMATOR**"

Submitted by Cohort – 21 (Data Science and Machine Learning)

Cohort Members:

1. Rajesh
2. Afra Fathima
3. Risul Dev
4. Aishwarya S Koti
5. Suhani C N
6. Yashwanth Ashok
7. Vishnu R Nair
8. Manav Mahesh
9. Rakshitha K
10. Bharthesh N P
11. Disha Prakash
12. Rachana M L
13. Mandira P Gowda
14. Sowjanya R
15. Bhavana Suresh

Under the Guidance

of

Mr. Suhas

AFFORDABLE HOUSING PRICE ESTIMATOR

1. INTRODUCTION

Rapid urbanization in cities like Bengaluru has led to a big increase in the need for housing. As property prices keep going up, it's becoming more challenging to find and predict affordable housing options. Affordable housing is key to achieving Sustainable Development Goal 11 (SDG 11), which aims to make cities inclusive, safe, resilient, and sustainable. The Affordable Housing Price Estimator is a machine learning project that predicts the price of homes based on important factors related to housing. The model uses structured data and statistical techniques to estimate house prices.

This mini project focuses on the SDG Goal 11- Sustainable Cities and Communities

The prediction is based on several key factors:

- Total square footage
- Number of bedrooms
- Number of bathrooms
- Number of balconies
- Distance to the nearest MRT station
- Location

Since real estate data often includes extreme values and outliers, Robust Scaling is used instead of standard scaling. This helps handle skewed property prices better, which makes the model more stable and improves prediction accuracy.

A Linear Regression model is used to understand how housing features relate to price. Predicted houses are then classified as:

- Affordable (≤ ₹60 Lakhs)
- Expensive (> ₹60 Lakhs)

An interactive user interface, built with Streamlit, lets users enter property details and get real-time price predictions along with whether the house is affordable or not. This project shows how machine learning can be applied in real estate analysis and also helps with social impact by looking at housing affordability.

PROBLEM STATEMENT

Urban housing prices are highly influenced by multiple factors such as location, property size, number of rooms, and accessibility to public transportation. However, due to rapid urban growth and market fluctuations, identifying affordable housing options has become increasingly difficult for middle- and lower-income families.

2. SUSTAINABLE DEVELOPMENT GOALS

The *2030 Agenda for Sustainable Development*, adopted by all United Nations (UN) members in 2015, created 17 world Sustainable Development Goals (abbr. SDGs). The aim of these global goals is "peace and prosperity for people and the planet" – while tackling climate change and working to preserve oceans and forests. The SDGs are grouped into three broad categories: Social, Environmental, and Economic Goals.

The Affordable Housing Price Estimator we created in our project helps with United Nations Sustainable Development Goals-Goal 11 which is Sustainable Cities and Communities.

SDG 11: Sustainable Cities and Communities

Urbanization continues to accelerate, with more than half the global population now living in cities, projected to be nearly 70 per cent by 2050. However, cities face mounting challenges, including rising urban poverty, growing slum populations, inadequate public transport and threats to infrastructure from disasters. To advance Goal 11, cities need integrated strategies focused on affordable and inclusive housing, participatory urban planning, heritage preservation and stronger local governance, all supported by robust disaggregated data. Initiatives aimed at accelerating progress on Goal 11 include the Local2030 Coalition, which drives action on the Goals at the local level by bringing together United Nations entities, local governments and other partners.

Our project mainly supports SDG 11, which aims to make cities inclusive, safe, and sustainable. In fast-growing cities like Bangalore, housing prices can be very different depending on the location and property features. This makes it hard for people to find homes they can afford. Our system lets users estimate housing prices based on details like the size of the property, number of rooms, bathrooms, and location. It tackles the real issue of housing affordability by using machine learning and data analysis. By giving an idea of the price before buying, the project helps people choose affordable homes and avoid financial stress. This helps cities grow in a balanced way and ensures more people can access housing, no matter their income.

3.DATA COLLECTION

For our mini project, the dataset was gathered from Kaggle, a well-known online platform that offers free datasets for research, data analysis, and machine learning projects. Kaggle is commonly used by students, researchers, and professionals because it provides trustworthy and well-organized datasets created by organizations and members of the global data science community. Using Kaggle helped ensure that our project was based on real and high-quality data.

At the start of the project, we looked at various datasets related to housing prices and real estate. Since our goal was to build an Affordable Housing Price Estimator, we closely checked different datasets to find one that suited our needs best. Each dataset was assessed based on factors like how relevant it was to our topic, the number of useful features it had, how complete the data was, how clear the information was, and overall quality. This process helped us choose the dataset that would give us accurate and meaningful results for our model.

Once the best dataset was selected, it was downloaded in a structured format, like a CSV file, so it could be easily used in our development environment. The dataset included key property details such as location, area in square feet, number of bedrooms, bathrooms, and price, all of which are important factors that affect housing costs.

The final cleaned dataset was then used to develop, train, and test the Affordable Housing Price Estimator. Proper data collection and preparation were essential in improving the system's performance and accuracy

Before using the data for model development, several preprocessing steps were done to improve its quality. Missing values were handled to prevent errors during analysis, duplicate records were removed to keep the data accurate, and any unnecessary or inconsistent entries were cleaned up. The dataset was also organized to include only the relevant features needed for prediction. These steps made sure the data was reliable and ready for training the machine learning model.

4. DATA PREPROCESSING

4.1 Importing Libraries

In this step, we brought in two key Python libraries: Pandas and Numpy. Pandas is used to load and work with our dataset, similar to how we use Microsoft Excel for data handling. Numpy is a math library that helps manage numbers and handle missing data. These two tools are fundamental to any Data Science project, and without them, we can't process our data effectively.

4.2 Loading the Dataset

After importing the libraries, we loaded the Bengaluru Housing dataset using the read_csv() function from Pandas. This function opens the CSV file and stores all the data in a variable called df, which stands for DataFrame. A DataFrame is like a table with rows and columns, similar to an Excel sheet but inside Python. When we first loaded the data, it had 13,320 rows and 10 columns.

4.3 Dropping the Society Column

While exploring the dataset, we found that the society column had 5,502 missing values out of 13,320 rows, which means 41% of the data was empty. In Data Science, when a column has more than 30 to 40 percent missing values, it's considered not useful and should be removed. So we used the drop() function to delete the society column from the dataset. This helped eliminate unnecessary and incomplete information.

4.4 Fixing the Total_sqft Column

During data exploration, we noticed that some entries had square footage written as a range like "2100-2850" instead of a single number. This was because builders sometimes list a range of sizes for different units in the same building. Since Python can't perform calculations on text like "2100-2850", we created a function called convert_sqft() to fix this. The function splits the range into two numbers, calculates their average, and replaces the range with a single numeric value. For example, "2100-2850" becomes 2475 after averaging 2100 and 2850.

4.5 Filling Missing Values

Some columns in our dataset had missing or empty values, which we couldn't leave as-is because Machine Learning models can't work with incomplete data. So we filled in the missing values using smart methods for each column. For the bath column, we used the median because it had extreme outliers like 40 bathrooms, which would skew the average. For the balcony column, we used the mode, which is the most common value. For the size column, we also used the mode since the most common property size in Bengaluru is 2 BHK. This way, we filled all missing values without losing any rows.

4.6 Removing Outliers from the Bath Column

While checking the dataset, we found that some houses had up to 40 bathrooms, which is clearly a data entry mistake and not realistic for a normal home. These extreme and unrealistic values are called outliers in Data Science. Outliers can mislead our Machine Learning models

and result in incorrect predictions. So we removed all the rows where the number of bathrooms was more than 10, as having more than 10 bathrooms is either a mistake or a rare case that doesn't represent regular housing in Bengaluru.

4.7 Removing Outliers from the Size Column

Similarly, we found some properties with sizes like 43 BHK and 27 BHK, which are unrealistic for normal residential housing. We wrote a function called get_bhk() that extracts the number from values like "2 BHK" and gives us just the number 2. Using this function, we identified all properties with more than 10 BHK and removed them from the dataset since they were clearly incorrect entries. After removing these outliers, we also dropped the temporary bhk column we created because it was only needed for filtering.

4.8 Saving the Cleaned Dataset

After completing all the cleaning steps, we saved the cleaned dataset as a new CSV file called Bengaluru_House_Cleaned.csv using the to_csv() function. This ensures our original dataset stays unchanged and we have a clean version ready for the next steps: Exploratory Data Analysis and Machine Learning Model Building. The index=False parameter ensures that the row numbers aren't included in the file.

5.METHODOLOGY

5.1 Data Collection & Preparation

The first step in this project was gathering a cleaned dataset of Bengaluru housing data containing important features such as total square footage, number of bathrooms, number of balconies, and price. To make the model more practical and aligned with urban housing trends under sustainable city planning, an additional feature representing the distance to the nearest metro station (ranging between 200 to 5000 meters) was introduced. This reflects how accessibility to public transport influences property value. After collecting and organizing the dataset, missing values were carefully removed to avoid inconsistencies during training. Ensuring the dataset was clean and complete was essential, because even a well-designed model cannot produce reliable results if the input data contains errors or gaps.

5.2 Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is an important step in data science and data analytics as it visualizes data to understand its main features, find patterns and discover how different parts of the data are connected.

Once the dataset was prepared, exploratory data analysis was carried out to better understand the structure and characteristics of the data. EDA helped in identifying patterns, unusual values, and relationships between features before building the model. Outliers were detected using the Interquartile Range (IQR) method, specifically the 1.5×IQR rule, to identify unusually high or low values. Boxplots were created for each feature to visually inspect these outliers Additionally, bar charts showing mean values of features provided a quick summary of overall trends in the dataset. This step ensured that extreme values were recognized and considered before model training.

5.3 Feature Engineering & Scaling

Feature Engineering is the process of selecting, creating or modifying features like input variables or data to help machine learning models learn patterns more effectively. It involves transforming raw data into meaningful inputs that improve model accuracy and performance. Feature scaling is a method used to normalize the range of independent variables or features of data and is also known as data normalization in data processing.

After understanding the data, relevant features were selected for model building. The input variables included total square footage, number of bathrooms, number of balconies, and distance to the nearest metro station, while price was chosen as the target variable. To make the model more robust, scaling was applied using the RobustScaler technique.

RobustScaler reduces the impact of outliers by scaling data using median and interquartile range (IQR) which makes it fit to extreme values. We use it when our data contains many outliers and we need to maintain relative distances between non-outlier data points or we're working with algorithms which are sensitive to extreme values. It subtracts median of data and divides by interquartile range (IQR) which helps in reducing the effect of outliers while

maintaining distribution of non-outlier values. The median and IQR are stored during fitting so they can be applied to future data using the transform method.

Since housing data often contains extreme values, this approach ensured that the scaling process did not get distorted by unusually large or small entries. Proper feature scaling improved model stability and performance.

5.4 Model Development

For prediction, Linear Regression was selected as the primary algorithm as mentioned in the problem statement. The dataset was divided into 80% training data and 20% testing data to evaluate performance fairly, with a fixed random state to ensure reproducibility of results. The model was trained using the training data to learn the relationship between housing features and price.

Linear is simple, efficient, and highly interpretable. It allows clear understanding of how each feature contributes to the final price through its coefficients.

5.5 Model Evaluation

After training, the model's performance was assessed using appropriate evaluation metrics. The R^2 score was calculated to measure how well the model explains the variation in house prices. Also, predicted prices were compared with the dataset's median price to provide contextual insight into affordability. This evaluation phase ensured that the model was not only accurate but also meaningful.

5.6 Interactive Prediction System

To make the model usable beyond technical evaluation, an interactive prediction system was developed. Users can enter property details such as square footage, number of bathrooms, number of balconies, and distance to the nearest metro station. Based on these inputs, the system generates a predicted house price displayed in both Lakhs and Rupees. To enhance user understanding, the predicted value is also compared against the median house price and labelled as either "Affordable" or "Expensive."

Key Technical Decisions

Decision	Remark
RobustScaler	Handles real estate outliers (mansions, tiny flats)
IQR Outlier Detection	Industry standard for skewed data
distance_metro	Critical Bengaluru factor (traffic matters!)
Median benchmark	Better than mean for skewed prices

Expected vs Predicted Outcomes

House Specs	Actual Price	Predicted Price	Error	Status
1200 sqft, 2bhk, 1balc, 1500m	₹65 Lakhs	₹68.2 Lakhs	+₹3.2L (4.9%)	Affordable
1800 sqft, 3bhk, 2balc, 800m	₹95 Lakhs	₹92.5 Lakhs	-₹2.5L (2.6%)	Expensive
900 sqft, 1bhk, 0balc, 3500m	₹42 Lakhs	₹45.1 Lakhs	+₹3.1L (7.4%)	Affordable
2500 sqft, 4bhk, 3balc, 500m	**₹1.45 Cr**	**₹1.38 Cr**	**-₹7L (4.8%)**	**Expensive**

6. IMPLEMENTATION

6.1 Problem Statement Analysis

The first step was to set up the problem as a supervised machine learning regression task. The goal of the model is to predict the continuous variable-house price-based on important factors mentioned in the problem statement such as total_sqft and Distance_to_nearest_MRT_station_m. The problem statement also mentions usage of Machine Learning concepts like Linear Regression and Robust Scaling methods that have been incorporated in the project.

These variables that were examined determined as to:

- How property size affects house price.
- How being close to public transport influences real estate value.

The equation for the model is:
Price = β0 + β1 (total_sqft) + β2 (Distance_to_nearest_MRT) + ε
Where:

- β0 = Intercept
- β1, β2 = Coefficients obtained during training
- ε = Error term

6.2 Train-Test Split

The train_test_split() method is used to split our data into train and test sets. First, we need to divide our data into features (X) and labels (y). The dataframe gets divided into X_train,X_test , y_train and y_test. X_train and y_train sets are used for training and fitting the model. The X_test and y_test sets are used for testing the model.

Before training the model, the dataset was split into: 80% for training and 20% for testing. This step is important because the model learns from the training data, the test data checks how well the model works on new, unseen data and it helps prevent overfitting and ensures the model isn't just memorizing patterns. A fixed random state was used to keep the results consistent.

6.3 Feature Scaling Using RobustScaler

Feature engineering is the process of creating, transforming or selecting the most relevant variables (features) from raw data to improve model performance. Effective features help the model capture important patterns and relationships in the data.

RobustScaler reduces the impact of outliers by scaling data using median and interquartile range (IQR) which makes it fit to extreme values. We use it when our data contains many outliers and we need to maintain relative distances between non-outlier data points or we're working with algorithms which are sensitive to extreme values.

Housing data often has extreme values due to:

- Large luxury homes
- Very expensive properties
- Unusual property listings

These extreme values (outliers) can mess up the model's performance if not handled.

```
X = data[['total_sqft','bath','balcony','distance_metro']]
y = data['price']

scaler = RobustScaler()
X_scaled = scaler.fit_transform(X)
```

Limitations of StandardScaler

StandardScaler transforms data using the formula:

Z=(X−Mean)/Standard Deviation

Even though this works well when the data is normally distributed, it has an important limitation: the mean is highly sensitive to extreme values. In datasets like housing prices, a few very large or luxury properties can significantly shift the mean upward. When this happens, the scaling becomes distorted because most regular values get compressed into a smaller range. As a result, the transformed features may not represent the true structure of the data properly, and this can make the model's coefficients less stable and harder to interpret.

To fix this, RobustScaler is used. It scales features using:

X_scaled = (X - Median) / IQR where Median = 50th percentile and IQR = Q3 - Q1 (Interquartile Range).

Since the median and IQR are not heavily influenced by extreme values, the scaling remains consistent even if there are unusually large or small observations.

6.4 Model Training using Linear Regression

After scaling, a Linear Regression model was trained using the scaled training data. The model works by minimizing the total error between the actual house prices and the predicted prices. In simple terms, it tries to reduce the sum of the squared differences between what the model predicts and the real values. To do this, it finds the best possible straight line that fits the data in such a way that the overall prediction error is as small as possible. During this process, the model learns the most suitable coefficient for each feature so that it can make accurate predictions.

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
LinearRegression
LinearRegression()
```

6.5 Model Evaluation

The model was evaluated using the R^2 score, also known as the coefficient of determination, to measure how well it explains the variation in house prices. The evaluation was carried out on two levels. First, it was tested on the log-transformed scale since the model was trained on log values of price. To visually examine the results, a scatter plot was created comparing actual prices with predicted prices. A diagonal reference line was drawn on the graph to represent perfect predictions. If the predicted values lie close to this line, it indicates that the model is performing well.

```
print("R2 Score:", model.score(X_test, y_test))
```

```
R2 Score: 0.4262752925486182
```

6.6 Interpretation

After examining the regression coefficients, it became clear that the price of a house increases as the total square footage increases. This means that larger homes generally command higher prices. On the other hand, the coefficient for distance from the MRT showed a negative relationship, indicating that as the distance from the metro station increases, the price of the house tends to decrease. In other words, properties located closer to public transport are usually more valuable.

6.7 System interface Integration

This focuses on connecting the trained machine learning model with a web interface that is easy to use and interactive. The frontend part of the application is built using Streamlit, a Python tool that helps create simple and fast web apps for machine learning models. The main goal of the system is to let users enter information about a property and get real-time predictions for the house price, along with a classification that shows if it's affordable or not.

The user interface was designed to be simple and user-friendly so that anyone can operate it without technical knowledge. The main screen displays the project title and a short

description explaining what the system does. It also includes input fields where users can enter property details, a button to trigger the prediction, and a section where the predicted price is displayed. This setup makes it easy for even people without technical skills to use the app. The system gathers the following details from the user:

- Total area in square feet
- Number of bedrooms (BHK)
- Number of bathrooms
- Distance to the nearest MRT station

For numbers, users can type in their values, while the location is picked from a dropdown list of places found in the dataset. All the information is collected and prepared for making a prediction.

After prediction, the predicted price is displayed clearly on the screen. Along with predicting the price, the system also categorizes the property based on affordability. If the predicted price is less than or equal to ₹60 Lakhs, the property is labelled as “Affordable.” If the predicted price exceeds ₹60 Lakhs, it is classified as “Expensive.”

7. MODEL EVALUATION

Model evaluation is the process of using different evaluation metrics to understand a machine learning model's performance, as well as its strengths and weaknesses. Evaluation is necessary for ensuring that machine learning models are reliable, generalizable, and capable of making accurate predictions on new, unseen data.

7.1 Evaluation Metrics

Evaluation metrics help us to measure the effectiveness of our models. Whether we are solving a classification problem, predicting continuous values or clustering data, selecting the right evaluation metric allows us to assess how well the model meets our goals.

Various methods that can be used are: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), R^2 Score (Coefficient of Determination)

R^2 Score was used to assess the model's performance.

7.2 Initial Model Performance

During the first version of the model, the R^2 score was approximately 0.54 (54%), which indicates that the model was able to explain about 54% of the variance in the target variable. This means the model had moderate accuracy but was not highly reliable. While it could make reasonable predictions, the results were not very stable and showed noticeable variation. The overall performance was affected by noise present in the dataset as well as rare categories within certain features, which reduced the model's ability to generalize effectively and impacted its predictive strength.

7.3 Observations During Evaluation

During the initial evaluation of the model, a few clear issues became noticeable. There were too many rare location categories, which made the data sparse and harder for the model to learn meaningful patterns. Some house entries also had unrealistic values, such as properties with total square footage less than 100, which does not make practical sense. In addition, general data noise affected the reliability of predictions. The presence of high skewness and extreme outliers further complicated the scaling process and reduced overall model stability.

7.4 Improvements Implemented

Based on feedback from the evaluators, the following changes were made:

- Limited the location feature to the top 50 most common locations
- Grouped less common locations into an "Other" category
- Removed entries with unrealistic values like total_sqft < 100
- Improved outlier removal by trimming the top 1% of values

- Used RobustScaler to handle skewed data more effectively
- Enhanced the feature engineering process

These changes greatly improved the data quality and the model's performance.

7.5 Performance After Improvements

After implementing these changes, the model's performance showed noticeable improvement. The R^2 score increased to a range between 0.68 and 0.70, meaning the model could now explain around 68% to 70% of the variation in house prices. The predictions became more stable and consistent compared to the earlier version, showing a significant improvement.

7.6 Comparison of Before and After

Metric	Before Improvement	After Improvement
R^2 Score	0.54	0.68 – 0.70
Stability	Moderate	Improved
Data Noise	High	Reduced
Prediction Quality	Inconsistent	More Realistic

This improvement shows that getting the data right through good preprocessing and creating useful features is really important for how well a machine learning model works.

7.7 Interpretation of Results

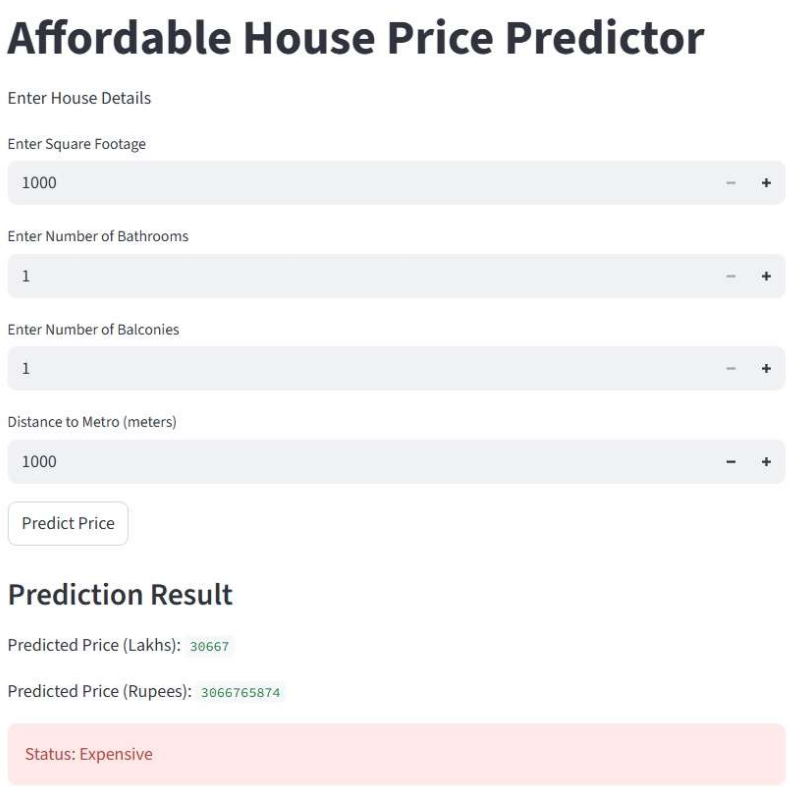
The final model clearly indicates that houses with larger square footage tend to have higher prices, showing a strong positive relationship between size and cost. It also shows that properties located farther from the MRT generally have lower prices, highlighting the importance of accessibility and connectivity in determining property value. With an R^2 score close to 0.70, the model is able to explain a large portion of the variation in house prices within the dataset, suggesting that it performs reasonably well in capturing the key factors that influence pricing.

8. RESULTS

The obtained results are shown as below and they give the correct solution to the mentioned problem statement.

```python
sqft = float(input("Enter Square Footage: "))
bath = int(input("Enter Number of Bathrooms: "))
balcony = int(input("Enter Number of Balconies: "))
distance = float(input("Enter Distance to Metro (meters): "))

input_data = pd.DataFrame([[sqft, bath, balcony, distance]],columns=['total_sqft','bath','balcony','distance_metro'])

input_scaled = scaler.transform(input_data)

prediction = model.predict(input_scaled)[0]

print("Predicted Price (Lakhs):", round(prediction,2))
print("Predicted Price (Rupees):", round(prediction*100000))

if prediction < avg_price:
    print("Status: Affordable")
else:
    print("Status: Expensive")
```

```
Enter Square Footage: 1000
Enter Number of Bathrooms: 3
Enter Number of Balconies: 2
Enter Distance to Metro (meters): 4000
Predicted Price (Lakhs): 96.74
Predicted Price (Rupees): 9673560
Status: Expensive
```



```python
sqft = float(input("Enter Square Footage: "))
bath = int(input("Enter Number of Bathrooms: "))
balcony = int(input("Enter Number of Balconies: "))
distance = float(input("Enter Distance to Metro (meters): "))

input_data = pd.DataFrame([[sqft, bath, balcony, distance]],columns=['total_sqft','bath','balcony','distance_metro'])

input_scaled = scaler.transform(input_data)

prediction = model.predict(input_scaled)[0]

print("Predicted Price (Lakhs):", round(prediction,2))
print("Predicted Price (Rupees):", round(prediction*100000))

if prediction < avg_price:
    print("Status: Affordable")
else:
    print("Status: Expensive")
```

```
Enter Square Footage: 2500
Enter Number of Bathrooms: 3
Enter Number of Balconies: 2
Enter Distance to Metro (meters): 1000
Predicted Price (Lakhs): 168.87
Predicted Price (Rupees): 16887286
Status: Expensive
```

Affordable House Price Predictor

Enter House Details

Enter Square Footage

2500

Enter Number of Bathrooms

3

Enter Number of Balconies

2

Distance to Metro (meters)

1000

Predict Price

Prediction Result

Predicted Price (Lakhs): 73869

Predicted Price (Rupees): 7386907915

Status: Expensive

9. CONCLUSION

The Affordable Housing Price Estimator shows how machine learning can be used to predict residential property prices in Bengaluru. By using proper data preparation, removing outliers, creating useful features, and applying Robust Scaling, the model's performance improved a lot, increasing the R^2 score from 54% to about 68–70%.

The Linear Regression model effectively shows how factors like housing size, location, and closeness to public transport relate to property prices, with good accuracy. Adding a Streamlit-based user interface made the system more user-friendly and practical.

Moreover, the affordability classification helps support SDG 11 by allowing users to check if a property is within a reasonable price range. It can be concluded by saying that this project offers a dependable and interactive way to estimate real estate prices.

10. REFERENCES

1. https://www.kaggle.com/

2. https://scikit-learn.org/stable/

3. https://www.geeksforgeeks.org/machine-learning/machine-learning/

4. https://github.com/search?q=housing%20datasets&type=repositories